Fundamentos de Informática

Funciones matemáticas en C++

La biblioteca <cmath>




```
#include <iostream>
#include <cmath>
using namespace std;
```

```
float r = sqrt(16.0f);  // r vale 4
```

- La línea **`#include <cmath>`** va una sola vez, al principio del programa
- Cada función recibe valores (argumentos) entre paréntesis y devuelve un resultado
- El resultado se guarda en una variable o se usa dentro de una expresión

Las tres funciones que van a usar

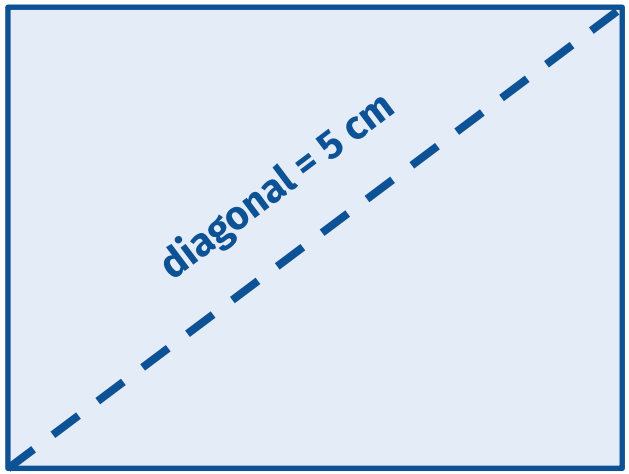
Función	Qué calcula	Ejemplo	Resultado
`sqrt(x)`	Raíz cuadrada de x	`sqrt(25.0f)`	`5`
`pow(x, y)`	x elevado a la y	`pow(2.0f, 3)`	`8`
`fabs(x)`	Valor absoluto de x (real)	`fabs(-3.5)`	`3.5`

El resultado se guarda en una variable `float`, igual que cualquier otro cálculo.

`<cmath>` tiene muchas más (`sin`, `cos`, `exp`, `log`, `round`...), pero para el TP alcanza con estas tres.

`sqrt`: la diagonal de una placa

Placa rectangular



$d = \sqrt{(base^2 + altura^2)}$

$= \sqrt{(16 + 9)} = \sqrt{25} = 5$ cm

- Es el teorema de Pitágoras
- Primero se resuelve lo que está dentro de la raíz; al final se saca la raíz

`sqrt` en código

```
float base = 4.0f;      // cm
float altura = 3.0f;    // cm

float suma = base * base + altura * altura;  // 25
float diagonal = sqrt(suma);                 // 5
```

Ojo con los paréntesis: `sqrt(a + b)` es la raíz de la suma; `sqrt(a) + b` es otra cosa.

- Primero se calcula lo de adentro de la raíz; **`sqrt`** se aplica al final
- La raíz de un número negativo da **`nan`** (not a number)
- Si hay muchos valores para sumar, se acumulan con un **`for`** (ejemplo al final)

Potencias: pow o multiplicar

`pow(x, y)`: cualquier exponente

```
float a = pow(2.0f, 3);     // 8
float b = pow(9.0f, 0.5f); // 3
float c = pow(x, 2);        // x²
```

- Elevar a 0.5 es lo mismo que la raíz cuadrada
- Necesita `<cmath>`

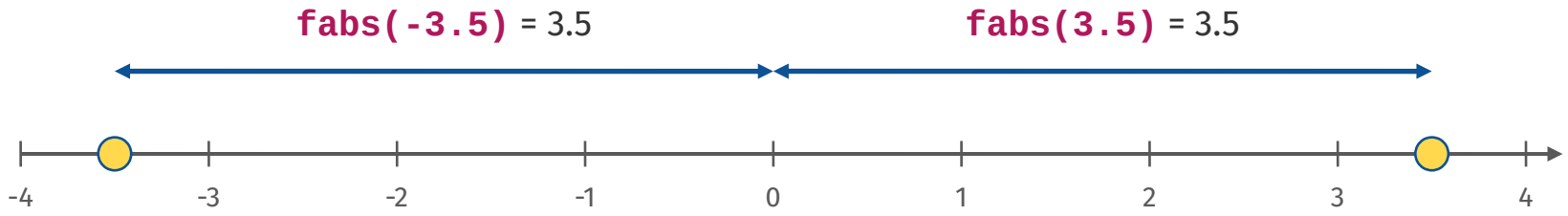
`x * x`: para elevar al cuadrado

```
float c = x * x;
float d = (a - b) * (a - b);
```

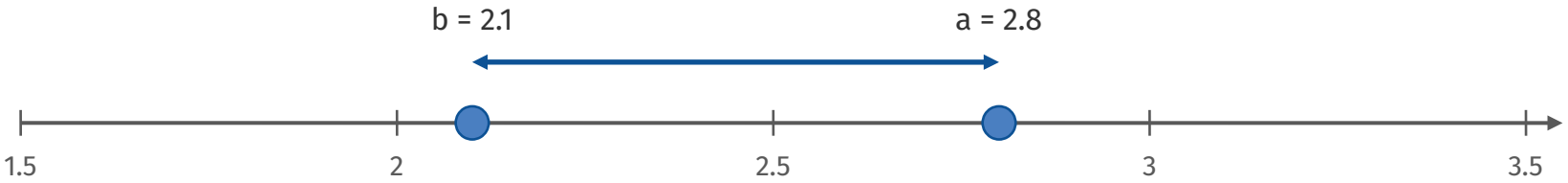
- Para cuadrados es más simple y no necesita `<cmath>`
- Con una resta, los paréntesis son obligatorios

Cuidado: `x^2` NO es x al cuadrado. Compila, pero hace otra operación (XOR entre bits): `3^2` da 1.

`fabs`: la distancia al cero



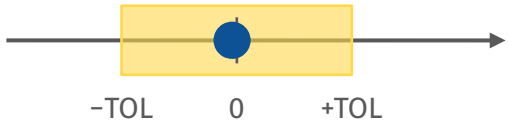
Distancia entre dos números: `fabs(a - b)` = `fabs(b - a)` = 0.7



Cálculo con tolerancia

Cuando un resultado sale de muchas cuentas con decimales, puede quedar apenas corrido del valor esperado. La tolerancia es el margen que aceptamos para darlo por bueno.

```
const float TOL = 0.0001f;
float suma = 0;   // desvíos de la presión
for (int i = 0; i < FILAS; i++) {
    suma = suma + (datos[i][0] - media);
}
if (fabs(suma) < TOL) {   // debería dar 0
    cout << "Da cero";
}
```



suma da −0.0000041: está dentro del margen, así que se acepta como 0.

¿Cuándo usarla?

- Para verificar un resultado que debería dar un valor justo (por ejemplo, 0)
- Cuando ese valor sale de muchas cuentas con decimales

¿Cuándo no hace falta?

- Para contar o recorrer índices
- Para ver cuál es mayor o menor (por ejemplo, buscar un máximo)

Tipos y división

Expresión	Resultado	Por qué
`7 / 2`	`3`	`int / int`: descarta decimales
`7.0f / 2`	`3.5`	Hay un real
`7 / 2.0f`	`3.5`	Hay un real
`(float) 7 / 2`	`3.5`	Conversión explícita

Regla práctica: antes de dividir, preguntarse si numerador y denominador son `int`. Si los dos lo son, el resultado pierde los decimales.

- Si el acumulador es `float`, dividirlo por un `int` da un resultado real
- Un `float` guarda unos 7 dígitos significativos; el resto se redondea

De la fórmula al código

En la fórmula	En C++	Cómo
Σ (sumatoria)	`for + acumulador`	El acumulador arranca en 0 antes del `for`
√x	`sqrt(x)`	Necesita `<cmath>`
x^2	`pow(x, 2)`	También se puede escribir `x * x`
|x|	`fabs(x)`	Necesita `<cmath>`
a / (b − 1)	`a / (b - 1)`	Respetar los paréntesis de la fórmula
x_i (i = 1…n)	`x[i]  (i = 0…n-1)`	Los índices empiezan en 0

Ejemplo: distancia entre dos pruebas

$d = \sqrt{\left(\Sigma_j \, (p1_j - p2_j)^2 \right)}$ distancia entre la prueba 1 y la prueba 2, sumando sobre las 4 variables

```
float suma = 0;
for (int j = 0; j < COLUMNAS; j++) {
    float dif = datos[0][j] - datos[1][j];
    suma = suma + pow(dif, 2);
}
float d = sqrt(suma);   // 5.84
```

Σ_j → **for** + acumulador

$(\dots)^2$ → **pow(dif, 2)**

$\sqrt{}$ → **sqrt(suma)**

Usa la matriz **datos** de **bombeo_matriz.cpp**: fila 0 = prueba 1, fila 1 = prueba 2.

Errores frecuentes

! Falta la biblioteca

Sin `#include <cmath>`, `sqrt` y `pow` no existen para el compilador.

! Usar ^ como potencia

`x^2` compila, pero no eleva al cuadrado. Usar `x * x`.

! Verificar sin margen

Un resultado con decimales puede no dar justo. Usar `fabs(a - b) < TOL`.

! Raíz de un negativo

`sqrt(-4.0)` da `nan` y contamina todos los cálculos siguientes.

! División entera

`1 / 2` da 0. Alcanza con que uno de los dos sea real.

! Paréntesis

`sqrt(a + b)` y `sqrt(a) + b` dan resultados distintos.

Para el TP: ¿dónde aparece cada función?

b) En la fórmula del desvío estándar muestral, ¿qué parte se resuelve con `sqrt`? ¿Qué se divide por n − 1?

d) Para verificar `A·v = λ·v` componente a componente, ¿cómo comparan dos valores que deberían ser iguales?

d) Si necesitan que un autovector tenga largo 1 (normalizarlo), ¿qué operaciones hacen?

e) El producto escalar debería dar 0: ¿qué condición escriben para «casi cero»?

Recordar: `#include <cmath>` al principio del programa.